
SQL - PostgreSQL

Prof. Sergio Salinas

TRABAJO PRÁCTICO INTEGRADOR

1. Ejercicios de manipulación de datos.

1. Inserta un nuevo empleado con sus datos personales en la tabla `employee`.
2. Registra un salario inicial para un empleado recién creado en la tabla `salary`.
3. Agrega un nuevo título para un empleado en la tabla `title`, indicando el periodo correspondiente.
4. Inserta un nuevo departamento en la tabla `department`.
5. Asigna un empleado a un departamento en la tabla `department_employee` con fechas válidas.
6. Registra un nuevo gerente para un departamento en la tabla `department_manager`.
7. Actualiza el salario actual de un empleado cerrando el intervalo anterior y creando uno nuevo.
8. Modifica el título activo de un empleado finalizando el intervalo anterior y añadiendo uno nuevo.
9. Cambia el nombre de un departamento existente en la tabla `department`.
10. Traslada un empleado de un departamento a otro registrando correctamente las fechas históricas.
11. Elimina un registro de salario específico de un empleado.
12. Borra un título histórico de un empleado sin afectar otros registros.
13. Elimina un departamento que no tenga asignaciones ni gerentes activos.
14. Actualiza las fechas de vigencia del departamento de un empleado corrigiendo un error de captura.
15. Modifica el intervalo de gestión de un gerente en un departamento.
16. Inserta múltiples salarios históricos para un empleado usando transacciones.
17. Registra varios títulos históricos para un empleado garantizando que no haya traslapes.
18. Inserta varios empleados nuevos y sus asignaciones iniciales a departamentos.
19. Marca el fin de la asignación de un empleado a un departamento prolongando o acortando el periodo.
20. Asigna un nuevo gerente a un departamento finalizando el periodo del anterior.
21. Actualiza el apellido de un empleado por corrección administrativa.

22. Cambia la fecha de contratación de un empleado.
23. Elimina la asignación de un empleado a un departamento en un periodo específico.
24. Inserta un nuevo periodo salarial para un empleado con efecto retroactivo.
25. Actualiza el `dept_name` de varios departamentos en una sola instrucción.
26. Cambia el género registrado de un empleado por solicitud del mismo.
27. Elimina todos los títulos históricos de un empleado que se va de la empresa.
28. Crea una nueva asignación de gerente para un departamento a partir de una fecha futura.
29. Corrige el periodo de un título de un empleado asegurando que no se superponga con otro.
30. Inserta datos de prueba: tres empleados, cada uno con dos salarios, dos títulos y dos departamentos asignados.

2. Ejercicios para generar reportes.

1. Obtén la lista de empleados con su salario actual, su título actual y el departamento en el que están asignados actualmente, asegurando que si falta alguna de estas asignaciones aparezcan igualmente.
2. Muestra todos los departamentos con su gerente actual, incluyendo los departamentos que no tengan gerente asignado.
3. Recupera todos los empleados junto con todos los departamentos en los que han trabajado alguna vez y el intervalo respectivo, incluyendo empleados sin asignaciones.
4. Lista todos los empleados y todos los departamentos posibles (producto cartesiano), indicando mediante un LEFT JOIN si el empleado ha pertenecido a ese departamento.
5. Obtén la secuencia histórica de salarios y títulos de cada empleado alineados por fecha, mostrando pares de periodos coincidentes mediante un JOIN por intervalos.
6. Muestra para cada departamento todos los empleados que trabajaron allí durante el periodo en que un gerente específico estuvo asignado.
7. Lista todos los empleados y el departamento cuyo gerente haya sido alguna vez la misma persona, usando SELF JOIN con `department_manager`.
8. Genera un reporte que liste todos los empleados cuyo último título y último salario no correspondan al mismo rango temporal.
9. Muestra, para cada departamento, la lista completa de empleados que hayan tenido un título coincidente con un periodo de asignación al departamento.
10. Obtén un listado de departamentos que hayan tenido cambios de gerente en un rango de fechas dado, mostrando el gerente anterior y el siguiente en una misma fila mediante SELF JOIN.

3. Ejercicios para la gestión de usuarios.

1. Crea un nuevo usuario llamado **analista** con contraseña segura.
2. Crea un rol llamado **reportes** sin permisos especiales.
3. Asigna el rol **reportes** al usuario **analista**.
4. Otorga al rol **reportes** permisos de **SELECT** sobre la tabla **employee**.
5. Revoca los permisos de **SELECT** sobre la tabla **salary** del usuario **analista**.
6. Cambia la contraseña del usuario **analista** por una nueva.
7. Crea un usuario **gestor** que solo pueda conectarse desde **localhost**.
8. Crea un rol **admin_departamentos** con permisos para **INSERT**, **UPDATE** y **DELETE** sobre la tabla **department**.
9. Asigna el rol **admin_departamentos** al usuario **gestor**.
10. Revoca todos los permisos del rol **reportes** sobre la tabla **title**.
11. Otorga al usuario **analista** la capacidad de crear bases de datos.
12. Crea un rol **auditor** con permisos solo de lectura sobre todas las tablas de la base de datos.
13. Concede al usuario **auditor** permisos de conexión a la base de datos pero sin poder modificar datos.
14. Crea un usuario temporal **temporal_user** que solo pueda conectarse durante un mes.
15. Elimina el rol **reportes** y asegura que los usuarios que lo tenían asignado pierdan esos permisos.
16. Muestra todos los usuarios existentes y sus roles asociados en la base de datos.
17. Revoca la posibilidad de que **gestor** cree nuevas funciones en la base de datos.
18. Crea un rol jerárquico **supervisor** y asigna los roles **reportes** y **auditor** como roles hijos.
19. Cambia el propietario de la tabla **department_employee** al usuario **gestor**.
20. Configura un usuario **readonly_user** que solo pueda ejecutar **SELECT** sobre todas las tablas y vistas de la base de datos.

4. Ejercicios de funciones, procedimientos y triggers.

1. Crea una función que reciba el **employee_id** y devuelva el nombre completo del empleado.
2. Crea una función que calcule el salario promedio de un empleado durante todo su historial.
3. Crea una función que reciba un **department_id** y devuelva la cantidad de empleados actualmente asignados a ese departamento.
4. Crea una función que devuelva el título actual de un empleado dado su **employee_id**.

5. Crea una función que devuelva el empleado con el salario más alto en un departamento específico.
6. Crea un procedimiento que inserte un nuevo empleado y automáticamente registre su primer salario y título.
7. Crea un procedimiento que transfiera a un empleado de un departamento a otro, actualizando correctamente los intervalos de asignación.
8. Crea un procedimiento que actualice el salario de un empleado, cerrando el intervalo anterior y registrando uno nuevo.
9. Crea un procedimiento que cambie el gerente de un departamento y actualice el historial de gestión.
10. Crea un procedimiento que registre múltiples empleados de prueba con datos aleatorios para testing.
11. Crea un trigger que registre en una tabla `audit_salary` cualquier cambio en la tabla `salary`.
12. Crea un trigger que impida que se inserte un salario negativo en la tabla `salary`.
13. Crea un trigger que actualice automáticamente la fecha de fin de un título anterior al insertar un nuevo título para un empleado.
14. Crea un trigger que envíe un mensaje a un log cada vez que un empleado sea asignado a un departamento.
15. Crea un trigger que verifique que un gerente asignado a un departamento no sea el mismo que un empleado ya asignado en otro rol.
16. Crea una función que devuelva todos los empleados cuyo salario promedio supere un valor dado.
17. Crea un procedimiento que reasigne todos los empleados de un departamento cerrado a otro departamento predeterminado.
18. Crea un trigger que impida eliminar un departamento si todavía tiene empleados asignados.
19. Crea un trigger que automáticamente actualice un campo `last_modified` en `employee` cuando se modifique cualquier registro relacionado (salario, título o departamento).
20. Crea una función que genere un reporte histórico completo de salarios, títulos y asignaciones de un empleado, incluyendo intervalos de fechas.